

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
MÔ HÌNH HÓA HÀNH VI VỚI UML
ACTIVITY, SEQUENCE
VÀ STATE-CHART DIAGRAM

Mã môn học: CO3001

Giảng viên hướng dẫn: Cô Trần Thị Ngọc Trâm

Mã lớp: L02

Nhóm: K24 Warriors

Học kỳ: Học KỲ 261

TP. Hồ Chí Minh, tháng 10/2026

Mục lục

I	Activity diagram	3
1	Lý thuyết từ đầu	3
2	Ký hiệu của bước thực hiện và dữ liệu	4
3	Ký hiệu rẽ nhánh và hội tụ	5
4	Ký hiệu song song, trách nhiệm và kết thúc	6
5	Ký hiệu sự kiện	8
6	Cách vẽ và những lỗi thường gặp	9
II	Sequence diagram	10
1	Lý thuyết từ đầu	10
2	Ký hiệu của bên tham gia và thực thi	11
3	Ký hiệu thông điệp	12
4	Combined fragment: các khung điều khiển tương tác	14
5	Cách vẽ và những lỗi thường gặp	16
III	State-chart diagram	17
1	Lý thuyết từ đầu	17
2	Ký hiệu của trạng thái và chuyển trạng thái	18
3	Hành vi bên trong state	20
4	Ký hiệu lựa chọn, phân cấp và song song	21
5	Cách vẽ và những lỗi thường gặp	22

IV Ví dụ activity diagram: đăng ký khóa học	24
1 Bài toán và phạm vi	24
2 Sơ đồ hoàn chỉnh	25
3 Đọc luồng chính	26
4 Các nhánh và vòng lặp	26
5 Dữ liệu, pin và các sự kiện	27
6 Vì sao các ký hiệu được dùng như vậy?	27
V Ví dụ sequence diagram: đặt hàng và thanh toán	28
1 Bài toán và các thành phần	28
2 Sơ đồ hoàn chỉnh	29
3 Đọc phần đặt hàng	32
4 Đọc vòng lặp thanh toán	32
5 Đọc phần hoàn tất checkout	32
6 Mở rộng interaction được tham chiếu	33
7 Liên hệ với từng ký hiệu đã học	34
VI Ví dụ state-chart diagram: vòng đời đơn sản xuất	35
1 Đối tượng và các sự kiện	35
2 Sơ đồ tổng quan	36
3 Đọc vòng đời và các nhánh kết thúc	37
4 Tạm dừng và deep history	37
5 Mở rộng state Gia công	39
6 Mở rộng state Đóng gói	40
7 Tổng hợp ý nghĩa của ví dụ	41

I Activity diagram

1 Lý thuyết từ đầu

1.1 Activity diagram dùng để trả lời câu hỏi nào?

Activity diagram là biểu đồ mô tả **luồng công việc**: những việc cần thực hiện, thứ tự giữa chúng, các điều kiện rẽ nhánh và những phần có thể làm song song.

Khi mô tả một quy trình, ta không chỉ cần biết “có những bước nào”, mà còn cần biết “bước nào xảy ra trước”, “nếu điều kiện không đạt thì đi đâu”, “ai thực hiện” và “khi nào kết thúc”. Activity diagram đưa những quan hệ đó lên hình.

Ví dụ, việc đăng ký một khóa học gồm nhập thông tin, kiểm tra điều kiện, xử lý học phí và xác nhận. Đây là quy trình. “Mở màn hình đăng ký” chỉ là một bước, không phải toàn bộ quy trình.

1.2 Activity khác action như thế nào?

Activity là hành vi lớn đang xét. **Action** là một đơn vị công việc bên trong hành vi đó. “Xử lý đăng ký khóa học” có thể là activity; “Kiểm tra dữ liệu hồ sơ” là action.

Mức độ chia nhỏ phụ thuộc mục đích của hình. Một sơ đồ nghiệp vụ có thể gom cả kiểm tra dữ liệu thành một action. Một sơ đồ chi tiết hơn có thể tách kiểm tra trường bắt buộc và kiểm tra định dạng. Không có quy tắc rằng mỗi dòng code phải trở thành một action.

1.3 Cách hiểu luồng bằng token

Có thể hình dung **token** như dấu hiệu cho biết một luồng đã sẵn sàng đi tiếp. Action nhận điều kiện đầu vào cần thiết, thực hiện công việc rồi đưa luồng đến bước sau.

Decision chọn một đường cho token. Fork tách luồng thành các nhánh. Merge gom các đường lựa chọn. Join đồng bộ những nhánh cần đợi nhau.

Token ở đây là khái niệm của mô hình thực thi, không phải token đăng nhập. Với object flow, token có thể mang dữ liệu hoặc đối tượng. Không nhất thiết phải vẽ token lên hình; hiểu nó giúp phân biệt các nút điều khiển.

1.4 Đọc sơ đồ

Tìm điểm bắt đầu, đi theo mũi tên, đọc action và xem guard mỗi khi gặp nhánh. Nếu có nhiều lane, xác định ai chịu trách nhiệm cho action. Nếu có thanh fork hoặc join, không đọc chúng như một câu hỏi đúng-sai.

Hình thường dàn trên-dưới, nhưng chiều trang không quyết định nghiệp vụ. Mũi tên mới cho biết hướng luồng.

2 Ký hiệu của bước thực hiện và dữ liệu

2.1 Initial node: điểm bắt đầu



Hình 1: Chấm đen là điểm bắt đầu

Initial node là chấm tròn đen. Nó đưa luồng vào activity khi lần thực thi bắt đầu. Với các quy trình thông thường, một điểm bắt đầu rõ ràng là đủ.

Không đặt câu hỏi hoặc điều kiện trong chấm đen. Điều kiện xử lý thuộc decision hoặc guard ở phía sau.

2.2 Action: bước công việc

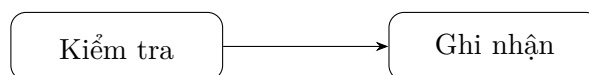
```
graph LR; Action(Kiểm tra hồ sơ);
```

Hình 2: Hình chữ nhật bo góc biểu diễn action

Đặt tên action bằng động từ và nội dung công việc: “Kiểm tra hồ sơ”, “Tính học phí”, “Giữ chỗ”. Action có thể là thao tác của người hoặc hệ thống.

“Đã xác nhận” là trạng thái, không phải tên action rõ ràng. Nếu muốn mô tả công việc, dùng “Xác nhận đăng ký”.

2.3 Control flow: luồng điều khiển

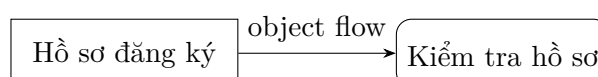


Hình 3: Mũi tên liền cho biết công việc tiếp tục ở đâu

Control flow thể hiện quan hệ điều khiển giữa các bước. Bước sau được phép tiến hành khi luồng đến đó và các điều kiện đầu vào của bước đã đủ.

Mũi tên này không mặc nhiên là lời gọi API hoặc dữ liệu truyền qua mạng. Nếu cần mô tả các thành phần gửi thông điệp cho nhau, dùng sequence diagram.

2.4 Object node và object flow: dữ liệu trong quy trình



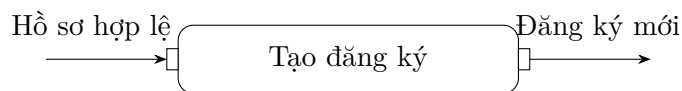
Hình 4: Dữ liệu đi vào action qua object flow

Object node là hình chữ nhật không bo góc, biểu diễn một đối tượng hoặc dữ liệu. Có thể ghi loại và trạng thái cần thiết, như **Registration [Pending]**.

Object flow chuyển dữ liệu đó giữa object node và action, hoặc giữa các đầu vào–đầu ra của action. Đường thường vẫn là mũi tên liền; phải nhìn các phần tử hai đầu để biết đây là dữ liệu, không chỉ là control flow.

“Nhập hồ sơ” là action; “Hồ sơ đăng ký” là dữ liệu. Không đổi hai thứ này cho nhau chỉ vì chúng cùng liên quan một biểu mẫu.

2.5 Pin: đầu vào và đầu ra của action



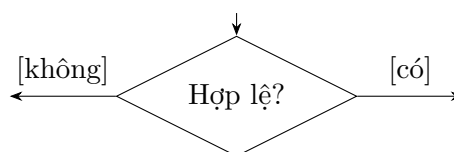
Hình 5: Các ô nhỏ ở cạnh action là input pin và output pin

Pin là ô vuông nhỏ gắn ở cạnh action, chỉ rõ đầu vào hoặc đầu ra. Input pin nhận dữ liệu; output pin cung cấp dữ liệu sau khi action hoàn tất.

Pin hữu ích khi cần biết action tiêu thụ và tạo ra gì. Không phải một cổng mạng, một nút giao diện hay một trạng thái.

3 Ký hiệu rẽ nhánh và hội tụ

3.1 Decision: chọn đường đi



Hình 6: Một luồng vào decision, nhiều đường lựa chọn đi ra

Decision chọn nhánh dựa trên điều kiện. Trong cách dùng đơn giản, mỗi lần đi qua sẽ chọn một nhánh phù hợp, không tự chạy tất cả nhánh.

Hình thoi có thể chứa câu hỏi ngắn. Điều kiện đầy đủ của mỗi đường nên nằm trên đường ra.

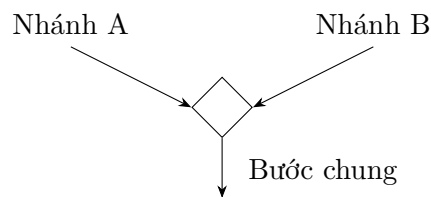
3.2 Guard: điều kiện của đường đi

Guard viết trong ngoặc vuông: [hồ sơ hợp lệ], [còn chỗ], [else]. Nó phải có thể được đánh giá đúng hoặc sai.

“Kiểm tra hồ sơ” là công việc, không phải guard. “Hồ sơ hợp lệ” mới là điều kiện.

Các guard nên loại trừ nhau và bao phủ những trường hợp cần xử lý. Nếu hai guard cùng đúng, nhánh được chọn có thể không xác định như người viết mong đợi. Nếu không guard nào đúng và không có nhánh còn lại, luồng có thể bị chặn.

3.3 Merge: gom các nhánh thay thế

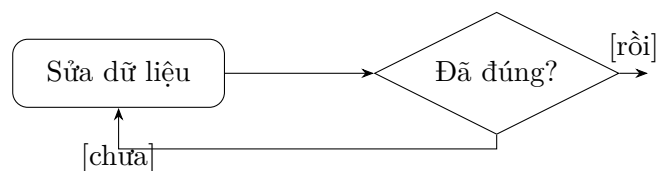


Hình 7: Merge không đợi tất cả nhánh

Merge gom những đường lựa chọn về một đường tiếp tục. Nhánh nào được chọn và đến merge thì đi tiếp; không cần nhánh không được chọn cũng đến.

Decision và merge cùng dùng hình thoi, nhưng hướng các luồng khác nhau. Đọc theo đường vào-ra chứ không chỉ nhìn hình dạng.

3.4 Vòng lặp



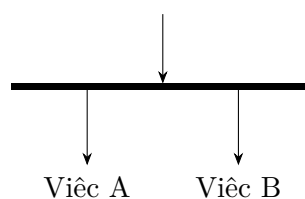
Hình 8: Đường quay lại kết hợp với guard tạo vòng lặp

Vòng lặp không có một ký hiệu riêng bắt buộc: nó được tạo bằng luồng quay lại và điều kiện. Phải biết lặp việc gì, lúc nào lặp tiếp và lúc nào thoát.

Nếu người dùng có quyền bỏ cuộc, cần mô tả cách dừng. Không vẽ một đường quay lại vô hạn rồi bỏ qua tình huống không còn muốn thực hiện.

4 Ký hiệu song song, trách nhiệm và kết thúc

4.1 Fork: tách luồng song song

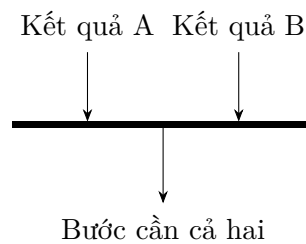


Hình 9: Fork khởi tạo các nhánh độc lập

Fork là thanh đậm với một luồng vào và nhiều luồng ra. Nó cho phép các nhánh được thực hiện đồng thời; không phải “chọn A hoặc B”.

Song song ở mức mô hình không bắt buộc dùng hai thread. Cách hiện thực có thể khác, miễn giữ đúng quan hệ độc lập và kết quả cần thiết.

4.2 Join: đồng bộ các nhánh

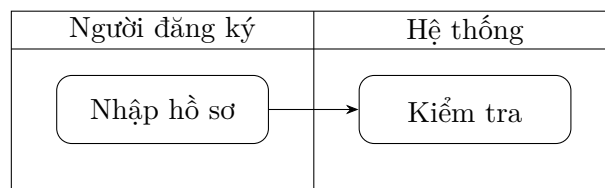


Hình 10: *Join đợi đủ những nhánh phải đồng bộ*

Join là thanh đậm có nhiều luồng vào và một luồng ra. Với cách dùng mặc định, nó đợi các nhánh cần thiết rồi mới cho bước sau tiếp tục.

Đừng dùng join để gom hai nhánh của decision: chỉ một nhánh được chọn thì nhánh còn lại không đến, nên bước sau có thể không bao giờ chạy. Ngược lại, merge không thay được join nếu bước sau thật sự cần hai kết quả.

4.3 Activity partition hay swimlane

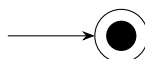


Hình 11: *Lane chia các action theo trách nhiệm*

Lane cho biết ai hoặc bộ phận nào thực hiện action. Nó có thể nằm dọc hay ngang tùy bố cục. Mũi tên được phép đi qua ranh giới lane.

Lane không mặc nhiên là một lớp, một server hoặc một bảng database. Action phải ở đúng vùng trách nhiệm: người nhập, hệ thống kiểm tra; không đảo hai bên cho tiện nối hình.

4.4 Activity final: kết thúc cả activity

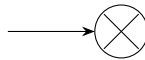


Hình 12: *Chấm đen trong vòng tròn ngoài là activity final*

Khi một luồng đến activity final, toàn bộ lần thực thi activity kết thúc. Các luồng còn đang chạy trong activity đó cũng bị dừng.

Dùng khi nghiệp vụ đang xét đã kết thúc, thành công hoặc không thành công. Ký hiệu không tự cho biết kết quả là tốt hay xấu; action và đường đi trước đó quyết định.

4.5 Flow final: chỉ kết thúc một luồng



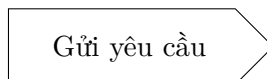
Hình 13: Vòng tròn có X là flow final

Flow final chỉ tiêu thụ luồng đi vào nó; các luồng khác có thể tiếp tục. Nếu không còn luồng nào hoạt động thì lần thực thi cũng có thể tự hoàn tất, nhưng không phải vì nút này ra lệnh dừng mọi nhánh.

Ví dụ, một nhánh phụ kết thúc không được làm dừng nhánh xử lý chính. Nếu cần bảo đảm cả hai nhánh đã làm xong, dùng join thay vì dùng activity final ở một nhánh.

5 Ký hiệu sự kiện

5.1 Send signal action

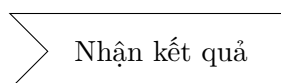


Hình 14: Action gửi tín hiệu có đầu nhọn

Send signal action gửi một tín hiệu bất đồng bộ. Bên gửi không cần chờ bên nhận xử lý xong để hoàn tất việc gửi.

Gửi yêu cầu thu học phí khác với thanh toán đã thành công. Muốn biết kết quả, phải có bước nhận hoặc truy vấn kết quả.

5.2 Accept event action



Hình 15: Action chờ sự kiện có cạnh trái lõm

Accept event action chờ một sự kiện phù hợp. Khi sự kiện chưa đến, luồng phía sau chưa được phép tiếp tục.

Phải nói rõ chờ sự kiện gì, từ đâu, và thuộc hồ sơ hoặc yêu cầu nào. Nhận một sự kiện bất kỳ không đủ để hoàn tất đúng một đăng ký cụ thể.

5.3 Time event



Đến một thời điểm hoặc hết một khoảng chờ

Hình 16: Đồng hồ cát biểu diễn chờ sự kiện thời gian

Sự kiện thời gian có thể gắn với một thời điểm tuyệt đối hoặc khoảng thời gian tương đối. “Đến ngày khai giảng” khác với “sau 15 phút kể từ khi giữ chỗ”.

Không dùng nó như hình trang trí cho một action “xử lý nhanh”. Nó có nghĩa luồng chờ điều kiện thời gian.

6 Cách vẽ và những lỗi thường gặp

Viết quy trình bằng câu ngắn trước, xác định người thực hiện, đặt action vào lane, rồi nối luồng chính. Sau đó thêm decision, guard và các nhánh còn lại. Chỉ thêm fork khi có những việc thật sự độc lập; thêm join nếu bước sau cần đủ kết quả.

Đọc thử từng đường từ đầu đến cuối. Một sơ đồ có đủ hình thoi và thanh đậm vẫn có thể sai nếu luồng không bao giờ thoát, nhánh từ chối lại đi vào bước ghi nhận thành công hoặc join đợi một nhánh chưa từng chạy.

Nhiều luồng đi thẳng vào một action không tự tương đương với merge. Action có thể cần đủ điều kiện đầu vào, gây đồng bộ ngoài ý muốn. Dùng nút gom nhánh rõ ràng.

Object flow phải mang dữ liệu phù hợp. Nếu action cần một hồ sơ hợp lệ, không nối một đối tượng “kết quả thanh toán” vào đó chỉ vì nó nằm gần trên hình.

II Sequence diagram

1 Lý thuyết từ đầu

1.1 Sequence diagram mô tả điều gì?

Sequence diagram là biểu đồ mô tả **sự tương tác giữa các bên theo thời gian**. Mỗi bên tham gia có một đường sống; các thông điệp nối từ bên gửi đến bên nhận.

Activity hỏi “quy trình đi qua những việc nào?”. Sequence hỏi “ai yêu cầu ai làm gì, khi nào, và nhận được kết quả gì?”. Một action trong activity có thể cần nhiều thông điệp trong sequence.

Ví dụ, “Xử lý thanh toán” có thể gồm dịch vụ đơn hàng gọi phiên thanh toán, phiên thanh toán gọi cổng thanh toán, cổng trả kết quả và dịch vụ lưu trạng thái. Không thể nhìn tên một action mà suy ra mọi tương tác đó.

1.2 Chọn mức mô tả

Ở mức system sequence, hệ thống có thể là một hộp đen: actor gửi yêu cầu, hệ thống trả kết quả. Ở mức design sequence, ta mở hộp đen để thấy các thành phần xử lý bên trong.

Chọn mức trước khi vẽ. Không để một participant “toàn bộ hệ thống” và một participant “dịch vụ đơn hàng” cùng thực hiện cùng một trách nhiệm mà không giải thích quan hệ.

1.3 Chiều thời gian và thứ tự

Thời gian đi từ trên xuống. Trái-phải chỉ phục vụ bố cục, không biểu thị thời gian hoặc mức quyền hạn.

Thứ tự các sự kiện trên một lifeline có ý nghĩa. Với những vùng song song hoặc những bên độc lập, không phải mọi sự kiện đều bị ép vào một thứ tự tuyệt đối trên toàn hình.

Khoảng cách giữa hai mũi tên không tự có đơn vị giây. Nếu cần giới hạn thời gian hoặc timeout, ghi điều kiện rõ ràng.

2 Ký hiệu của bên tham gia và thực thi

2.1 Participant và actor



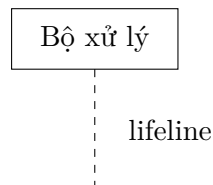
Hình 17: *Participant thường là ô tên; actor có thể dùng hình người*

Participant là bên tham gia tương tác: người dùng, đối tượng, thành phần xử lý, kho dữ liệu hoặc hệ thống ngoài. Actor là vai trò bên ngoài đang tương tác với hệ thống.

Có thể ghi `orderService:OrderService` để chỉ một đối tượng của lớp `OrderService`. Trong phân tích sơ bộ, một tên trách nhiệm rõ ràng cũng đủ nếu chưa chốt lớp.

Ký hiệu hình trụ có thể dùng cho kho dữ liệu. Một đối tượng nghiệp vụ không mặc nhiên là database. Các hình control hay boundary là cách bổ sung vai trò participant; dù đổi biểu tượng, ý nghĩa đường sống và thông điệp vẫn phải nhất quán.

2.2 Lifeline: đường sống

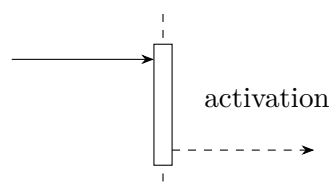


Hình 18: *Đường nét đứt dưới participant là lifeline*

Lifeline thể hiện sự tham gia của một bên trong tương tác đang xét. Nó kéo dài qua các thời điểm bên đó tồn tại hoặc tham gia.

Một lifeline không tương đương một thread. Một server có thể chứa nhiều đối tượng; một participant mức phân tích có thể được hiện thực bằng nhiều lớp.

2.3 Activation: khoảng đang thực hiện hành vi



Hình 19: *Thanh hẹp trên lifeline là execution specification*

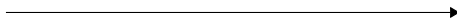
Activation biểu thị khoảng một hành vi đang được thực thi. Nó có thể bắt đầu khi nhận lời gọi và kết thúc khi công việc tương ứng hoàn tất.

Nó không biểu thị server đang bật hay ứng dụng đang đăng nhập. Nếu không vẽ activation, participant vẫn có lifeline; chỉ là hình không nhấn mạnh khoảng thực thi.

3 Ký hiệu thông điệp

3.1 Synchronous message: gọi đồng bộ

authorizePayment()



Hình 20: Thông điệp đồng bộ: đường liền, đầu mũi tên kín

Bên gọi cần chờ phần xử lý của bên nhận hoàn tất trước khi tiếp tục phần việc phụ thuộc kết quả. Ví dụ, cần biết có thanh toán thành công không trước khi xác nhận đơn.

Đồng bộ ở mức tương tác không có nghĩa giao diện phải khóa toàn bộ màn hình. Đây là quan hệ chờ kết quả, không phải chỉ dẫn viết code chặn luồng giao diện.

3.2 Asynchronous message: gửi bất đồng bộ

dispatch(orderId)



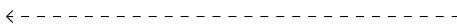
Hình 21: Thông điệp bất đồng bộ: đường liền, đầu mũi tên mở

Bên gửi không phải chờ công việc bên nhận làm xong mới tiếp tục. Gửi yêu cầu xử lý giao hàng có thể hoàn tất trước khi việc giao hàng thực sự được xử lý.

Bất đồng bộ không đồng nghĩa không cần kết quả. Kết quả có thể đến qua một sự kiện hoặc một thông điệp khác sau đó.

3.3 Reply message: trả kết quả

paymentStatus

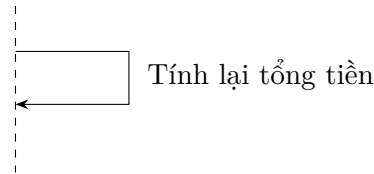


Hình 22: Thông điệp trả về thường dùng nét đứt

Reply trả kết quả cho lời gọi. Kết quả có thể thành công, từ chối hoặc lỗi. Nét đứt không có nghĩa “không quan trọng” và không dùng riêng cho ngoại lệ.

Có thể lược bớt reply hiển nhiên để hình gọn, nhưng nên giữ kết quả làm quyết định cho những bước tiếp theo.

3.4 Self-message: tự gọi

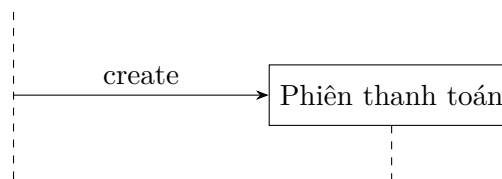


Hình 23: Mũi tên quay lại cùng lifeline là self-message

Self-message thể hiện bên đang xét thực hiện một hành vi nội bộ. Ví dụ, dịch vụ đơn hàng kiểm tra dữ liệu hoặc tính tổng tiền.

Nó không phải thông điệp giữa hai server. Cũng không cần thêm một self-message cho mọi phép gán trong chương trình.

3.5 Create message: tạo đối tượng



Hình 24: Lifeline của đối tượng mới bắt đầu tại lúc được tạo

Đối tượng mới không tồn tại từ đầu tương tác. Đầu lifeline được đặt tại thời điểm tạo, với thông điệp đi đến đầu đó.

“Tạo đơn hàng” có thể là tạo đối tượng hoặc ghi dữ liệu tùy mức thiết kế. Phải nói rõ đang mô hình hóa điều gì; không cứ có chữ “tạo” là bắt buộc thêm participant mới.

3.6 Destroy message: kết thúc tồn tại



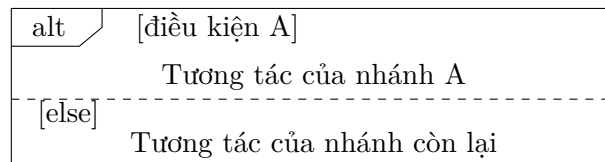
Hình 25: Dấu X ở cuối lifeline đánh dấu đối tượng bị hủy

Destroy kết thúc sự tồn tại của đối tượng được mô hình hóa. Không nối thông điệp sử dụng thông thường vào lifeline đã bị hủy trong cùng kịch bản.

Hủy một đăng ký hay đơn hàng về mặt nghiệp vụ không nhất thiết xóa đối tượng khỏi hệ thống. Bản ghi có thể vẫn được giữ để xem lịch sử. Trong ví dụ phần V, đối tượng bị hủy là phiên xử lý tạm, không phải đơn hàng.

4 Combined fragment: các khung điều khiển tương tác

4.1 Cấu tạo chung của fragment



Hình 26: Toán tử ở góc khung; guard và thông điệp nằm trong các vùng

Fragment bao một phần tương tác. Toán tử quyết định cách đọc vùng bên trong. Một đường phân cách trong khung chia các operand, tức những vùng tương tác của toán tử đó.

Guard là điều kiện; không phải một thông điệp. Chữ `alt` hay `loop` không phải tên một lớp xử lý.

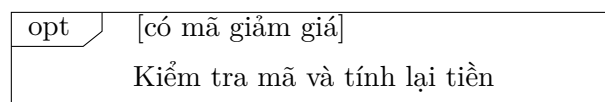
4.2 alt: chọn một khả năng

`alt` mô tả các khả năng thay thế nhau. Ví dụ: thanh toán thành công hoặc thất bại. Viết guard cho từng vùng và giữ chúng không chồng lấn khi cần lựa chọn xác định.

Nếu một vùng là `[thành công]`, vùng kia có thể dùng `[else]`. Không vẽ hai nhánh xong rồi lại mặc nhiên thực hiện thông điệp xác nhận thành công ở ngoài khung cho cả hai.

4.3 opt: tùy chọn

`opt` có một vùng được thực hiện nếu guard đúng; nếu sai, bỏ qua vùng này. Nó tương tự một nhánh “nếu có thì làm” không có xử lý thay thế.

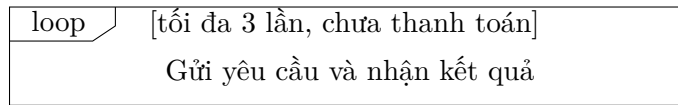


Hình 27: Một vùng tùy chọn trong `opt`

Nếu khi không có mã giảm giá còn phải thực hiện một tương tác khác, `alt` thường dễ diễn đạt hơn. Không nhầm `opt` với `extend` của use-case diagram; chúng ở hai loại sơ đồ và hai mức mô tả khác nhau.

4.4 loop: lặp tương tác

`loop` bao những thông điệp được lặp. Ghi điều kiện, giới hạn số lần hoặc cả hai.



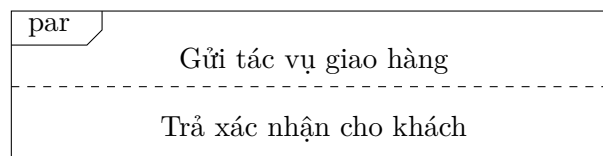
Hình 28: Loop cần điều kiện tiếp tục và điều kiện dừng

Ví dụ, thử thanh toán tối đa ba lần và dừng sớm khi thành công. “Ba lần” không có nghĩa luôn phải làm đủ ba lần nếu mục tiêu đã đạt.

Dừng chỉ viết loop rồi bỏ trống điều kiện. Người đọc cần biết vì sao công việc lặp và khi nào phần dưới khung được thực hiện.

4.5 par: các vùng song song

par cho phép tương tác trong các vùng được thực hiện song song hoặc xen kẽ. Thứ tự bên trong mỗi vùng vẫn được giữ.



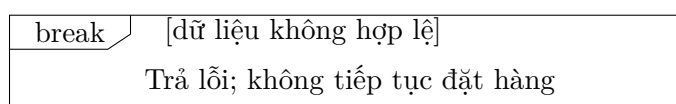
Hình 29: Par chứa các vùng tương tác độc lập

Không hiểu đường phân cách trong par như “chọn một trong hai”. Cả hai vùng có thể diễn ra trong cùng một kịch bản.

Nếu công việc B cần kết quả của A, chúng không độc lập để đặt vào hai vùng par mà không mô tả đồng bộ.

4.6 break: dừng phần tương tác còn lại

break mô tả tương tác thay thế khi guard đúng và bỏ qua phần còn lại của tương tác bao quanh trong phạm vi liên quan.

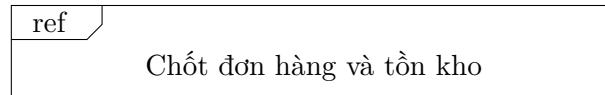


Hình 30: Break dùng cho trường hợp không thể tiếp tục tương tác

Guard sai thì bỏ qua break và tiếp tục phía dưới. Guard đúng thì thực hiện vùng break và không thực hiện phần còn lại đang được thay thế.

Nó không phải dấu X hủy đối tượng, cũng không phải nút flow final của activity.

4.7 ref: tham chiếu một tương tác khác

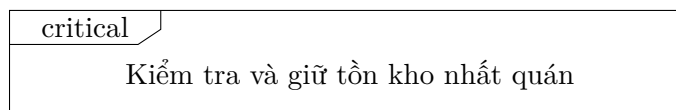


Hình 31: Ref đặt tên tương tác được trình bày ở một hình khác

Ref tránh lặp lại một đoạn phức tạp. Tên trong khung phải có nội dung tương ứng để tra cứu, không phải nhãn “xử lý tiếp” rồi không giải thích.

Khi mở hình được tham chiếu, các bên và dữ liệu trao đổi phải khớp với nơi gọi.

4.8 critical: vùng không cho xen kẽ



Hình 32: Critical đánh dấu yêu cầu không xen kẽ trên các lifeline liên quan

Critical mô tả vùng tương tác không được xen kẽ bởi những sự kiện khác trên các lifeline được bao trong vùng.

Khung này không tự tạo transaction, khóa database hay chống đặt trùng. Nó chỉ nói yêu cầu của mô hình. Cơ chế nhất quán vẫn phải được hiện thực phù hợp.

Khung **group** của PlantUML chỉ gom thông điệp theo một nhãn để dễ đọc; không có nghĩa nguyên tử như một transaction.

5 Cách vẽ và những lỗi thường gặp

Chọn một kịch bản và xác định participant theo trách nhiệm. Vẽ thông điệp mở đầu, những lời gọi cần thiết, kết quả trả về và các quyết định dựa trên kết quả đó. Thêm fragment sau khi luồng chính đã rõ.

Không coi mỗi dòng main flow là một participant. Participant là bên thực hiện; message là việc các bên trao đổi.

Không gửi một thao tác ghi dữ liệu xong rồi trả thành công trước khi biết đã ghi được. Không gọi một participant đã bị hủy. Không dùng mũi tên bất đồng bộ nếu bên gửi vẫn cần kết quả ngay trước khi làm bước tiếp theo mà chưa mô tả cách chờ kết quả.

Nếu hình dài, có thể chia trên nhiều trang nhưng phải giữ cùng participant, thứ tự và điều kiện; đây vẫn là một tương tác, không phải tự đổi nghiệp vụ giữa hai hình.

III State-chart diagram

1 Lý thuyết từ đầu

1.1 State chart mô tả điều gì?

State chart, hay state machine diagram, mô tả **vòng đời và phản ứng của một đối tượng**: đối tượng có thể ở trạng thái nào, sự kiện nào khiến nó chuyển trạng thái, và điều kiện nào cho phép chuyển.

“Đơn sản xuất đang tạm dừng” nói về trạng thái. “Nhân viên bấm nút tạm dừng” nói về sự kiện. “Lưu tiến độ” nói về hành động. Ba thứ này phải được tách ra trước khi vẽ.

1.2 Phải chọn một đối tượng

Đặt tên đối tượng ngay từ đầu, như một đơn sản xuất hoặc một phiên xử lý. Không trộn trạng thái của đơn hàng, tài khoản khách và cổng thanh toán vào cùng một máy trạng thái.

Một use case chỉ đọc dữ liệu không nhất thiết tạo vòng đời mới. Ngược lại, một máy trạng thái có thể liên quan nhiều use case cùng tác động lên một loại đối tượng.

1.3 State khác một bước công việc

Một state thường kéo dài trong một khoảng và ảnh hưởng những việc được phép làm. “Đang gia công” là state; “Kiểm tra hồ sơ” là công việc.

State chart không phải bản sao của activity. Không biến mọi action thành một state. Trạng thái của màn hình cũng không tự là trạng thái nghiệp vụ của đối tượng.

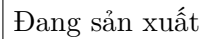
1.4 Cách đọc một chuyển trạng thái

Đọc state nguồn, sự kiện kích hoạt, guard, effect và state đích. Hỏi thêm “nếu sự kiện này đến khi đang ở state khác thì có được xử lý như vậy không?”.

Một máy trạng thái không chỉ nói các đường được phép đi; nó cũng giúp nhận ra những đường *không được phép*. Đơn đã hoàn tất không thể âm thầm quay về đang sản xuất nếu không có nghiệp vụ tạo một lần xử lý khác.

2 Ký hiệu của trạng thái và chuyển trạng thái

2.1 State: trạng thái



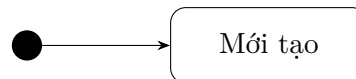
```
stateDiagram-v2
    state "Đang sản xuất" as S1
```

Hình 33: State có tên trong hình chữ nhật bo góc

State có thể chỉ có tên hoặc thêm một ngăn ghi hành vi. Tên nên mô tả tình trạng, không phải một thao tác bấm nút.

Cùng một tên “Completed” ở hai loại đối tượng không có nghĩa chúng dùng chung vòng đời.

2.2 Initial pseudostate



Hình 34: Initial xác định state đầu tiên

Chấm đen là điểm khởi đầu, không phải state nghiệp vụ cần lưu vào trường status. Trong cách dùng thông thường, nó có một đường ra để xác định trạng thái khởi tạo.

Đường ra từ initial thường không mang trigger hay guard. Nếu cần chọn trạng thái theo dữ liệu khởi tạo, chuyển đến choice để diễn đạt việc chọn.

2.3 Final state



Hình 35: Final đánh dấu hoàn tất phạm vi state machine đang xét

Ở mức máy trạng thái ngoài cùng, final là hoàn tất vòng đời được mô hình hóa. Bên trong composite, nó đánh dấu hoàn tất một region.

Final không yêu cầu xóa bản ghi. Nó cũng không có nghĩa “một công việc khác có thể tiếp tục” giống flow final trong activity.

2.4 Transition



Hình 36: Transition là đường chuyển có hướng giữa các state

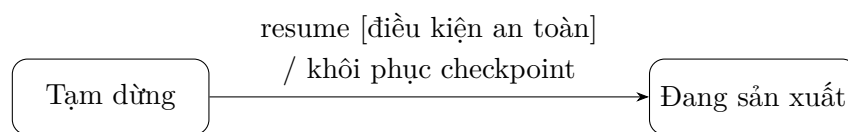
Transition biểu diễn khả năng rời state nguồn và sang state đích. Nó không phải control flow của một danh sách action và không phải message giữa hai lifeline.

Có một transition không có nghĩa lúc nào cũng được đi qua nó. Trigger, guard và state hiện tại quyết định.

2.5 Trigger, guard và effect

Nhãn thường có dạng:

event [guard] / effect.



Hình 37: Sự kiện kích hoạt, điều kiện cho phép và hành động khi chuyển

Trigger xác định sự kiện được tiếp nhận. **Guard** là điều kiện phải đúng khi xét transition. **Effect** là hành vi thực hiện khi transition được lấy.

Ví dụ, yêu cầu tiếp tục đến nhưng điều kiện an toàn chưa đạt thì không chuyển sang đang sản xuất. Guard không tự phát sinh yêu cầu tiếp tục.

Với transition ngoài thông thường, thứ tự là exit của state nguồn, effect của transition, rồi entry của state đích, sau khi transition đã được cho phép.

2.6 Time event và completion transition

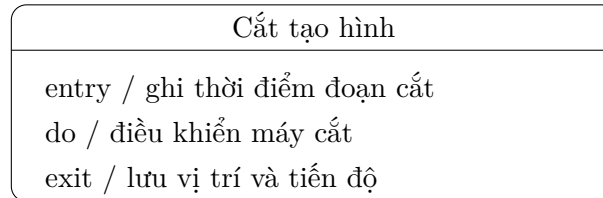
`after(30 phút)` là sự kiện thời gian tương đối. Cần xác định tính từ khi vào state nào, không chỉ ghi một con số trên mũi tên.

Một transition không có trigger có thể là completion transition: nó được xét khi state nguồn hoàn tất hành vi cần thiết. Không phải vì không ghi nhãn mà mọi điều kiện đều bị bỏ qua.

Ví dụ, composite đóng gói hoàn tất cả các vùng thì có thể chuyển ra “Hoàn tất đóng gói” mà không cần actor bấm thêm nút.

3 Hành vi bên trong state

3.1 Entry, do và exit



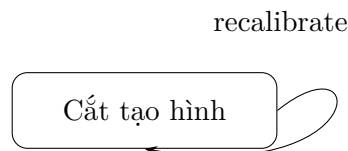
Hình 38: Ba loại hành vi của state

entry chạy khi vào state. **do** là công việc diễn ra trong thời gian ở state. **exit** chạy khi rời state.

Một do activity có thể kéo dài và bị ngắt khi state bị rời. Không gán mọi công việc vào entry: nếu chỉ xảy ra trên một đường chuyển cụ thể, effect của transition có thể hợp lý hơn.

Nếu quay lại state, entry có thể chạy lại. Phải biết việc ghi thời gian đang ghi thời điểm cả quy trình hay một đoạn xử lý mới, tránh ghi đè dữ liệu gốc ngoài ý muốn.

3.2 External self-transition

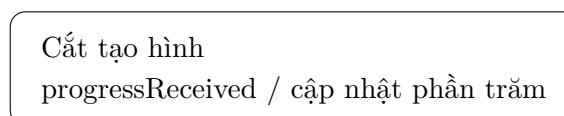


Hình 39: Self-transition ngoài rời rồi vào lại state

Mũi tên quay về chính state thường là external self-transition. Nó rời state và vào lại, nên có thể thực hiện lại exit và entry.

Ví dụ, hiệu chỉnh lại máy cắt làm kết thúc đoạn cắt hiện tại và bắt đầu đoạn xử lý mới với thông số mới. Nếu không muốn hành vi vào-ra này, cần internal transition.

3.3 Internal transition



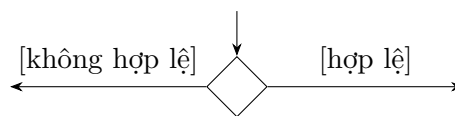
Hình 40: Internal transition được ghi bên trong state

Internal transition xử lý sự kiện mà không rời state. Ví dụ, bản tin tiến độ chỉ cập nhật phần trăm, không dừng máy và khởi tạo lại đoạn cắt.

Không có exit và entry của state do sự kiện nội bộ này. Đây là khác biệt về hành vi, không chỉ khác cách dàn đường vẽ.

4 Ký hiệu lựa chọn, phân cấp và song song

4.1 Choice

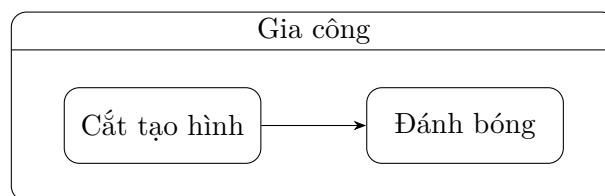


Hình 41: Choice chọn transition theo guard tại thời điểm đi qua

Choice là pseudostate hình thoi. Nó đánh giá guard để chọn đường chuyển; không phải một trạng thái “đang lựa chọn” cần lưu như status nghiệp vụ.

Hình thoi cũng xuất hiện trong activity, nhưng ở đây nó thuộc máy trạng thái của một đối tượng. Không nối các đối tượng khác nhau vào choice như đang vẽ luồng xử lý liên thành phần.

4.2 Composite state và substate

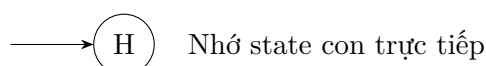


Hình 42: Composite chứa những state con trong cùng phạm vi

Composite state chứa các state con. Khi ở một state con, đối tượng cũng đang ở composite bao quanh nó.

Một transition từ khung composite ra ngoài có thể dùng chung cho các trạng thái con, chẳng hạn yêu cầu tạm dừng trong quá trình gia công. Không cần chép cùng một mũi tên từ mọi state con nếu mô hình phân cấp đã diễn đạt đúng.

4.3 Shallow history: H

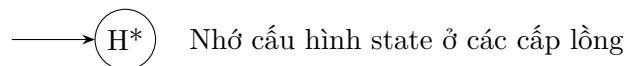


Hình 43: H là điểm nhớ nông

Shallow history nhớ state con trực tiếp đã hoạt động trước khi rời composite. Khi quay lại qua H, khôi phục state con đó; phần lồng sâu hơn thường được khởi tạo theo cách mặc định tương ứng.

Nếu composite chỉ có các state con đơn giản, H là đủ. History không phải audit log và không dùng để xem lại mọi thao tác đã xảy ra.

4.4 Deep history: H^*

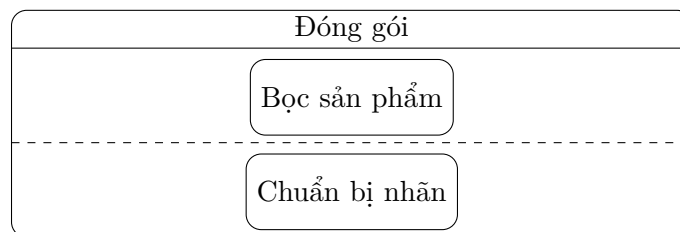


Hình 44: H^* là điểm nhớ sâu

Deep history nhớ cấu hình trạng thái sâu bên trong composite. Nếu trước khi tạm dừng đang ở “Đang sản xuất / Gia công / Đánh bóng”, H^* giúp quay lại đúng đoạn đó thay vì quay về bước đầu của gia công.

History nhớ *trạng thái điều khiển*. Muốn tiếp tục đúng dữ liệu và vị trí máy, còn cần checkpoint hoặc dữ liệu nghiệp vụ được lưu. H^* không tự lưu database.

4.5 Orthogonal regions: các vùng song song



Hình 45: Một composite có nhiều region hoạt động cùng lúc

Trong composite có nhiều orthogonal regions, đối tượng có thể đồng thời ở một state của mỗi region. Đây không phải “bọc hoặc chuẩn bị nhân”, mà là hai phần cùng hoạt động.

Mỗi region có thể có initial và final riêng. Hoàn tất một region không tự hoàn tất composite nếu region khác còn đang làm việc.

Không ghép state chart của nhiều đối tượng độc lập vào một khung rồi gọi đó là orthogonal regions. Các vùng phải thuộc cùng đối tượng hoặc bộ điều khiển đang được mô hình hóa.

5 Cách vẽ và những lỗi thường gặp

Chọn đối tượng, liệt kê state có ý nghĩa, rồi xác định sự kiện được chấp nhận ở từng state. Ghi guard và effect cho các chuyển cần giải thích. Sau đó mới thêm composite,

history hoặc vùng song song nếu nghiệp vụ cần.

Đừng tạo một state cho từng giá trị tiến độ. Từ 45% lên 46% thường là cập nhật thuộc tính trong cùng state, không phải hai trạng thái nghiệp vụ mới.

Đừng dùng một guard trơ như `[nhiệt độ cao]` rồi mặc nhiên cho rằng máy trạng thái biết khi nào cần xét nó. Phải có sự kiện dữ liệu mới, sự kiện thay đổi hoặc cơ chế phát hiện tương ứng.

State chart không tự giải quyết sự kiện xung đột. Nếu yêu cầu hủy và xác nhận hoàn tất đến gần nhau, giải pháp vẫn phải kiểm tra trạng thái và chỉ ghi nhận một kết quả hợp lệ.

IV Ví dụ activity diagram: đăng ký khóa học

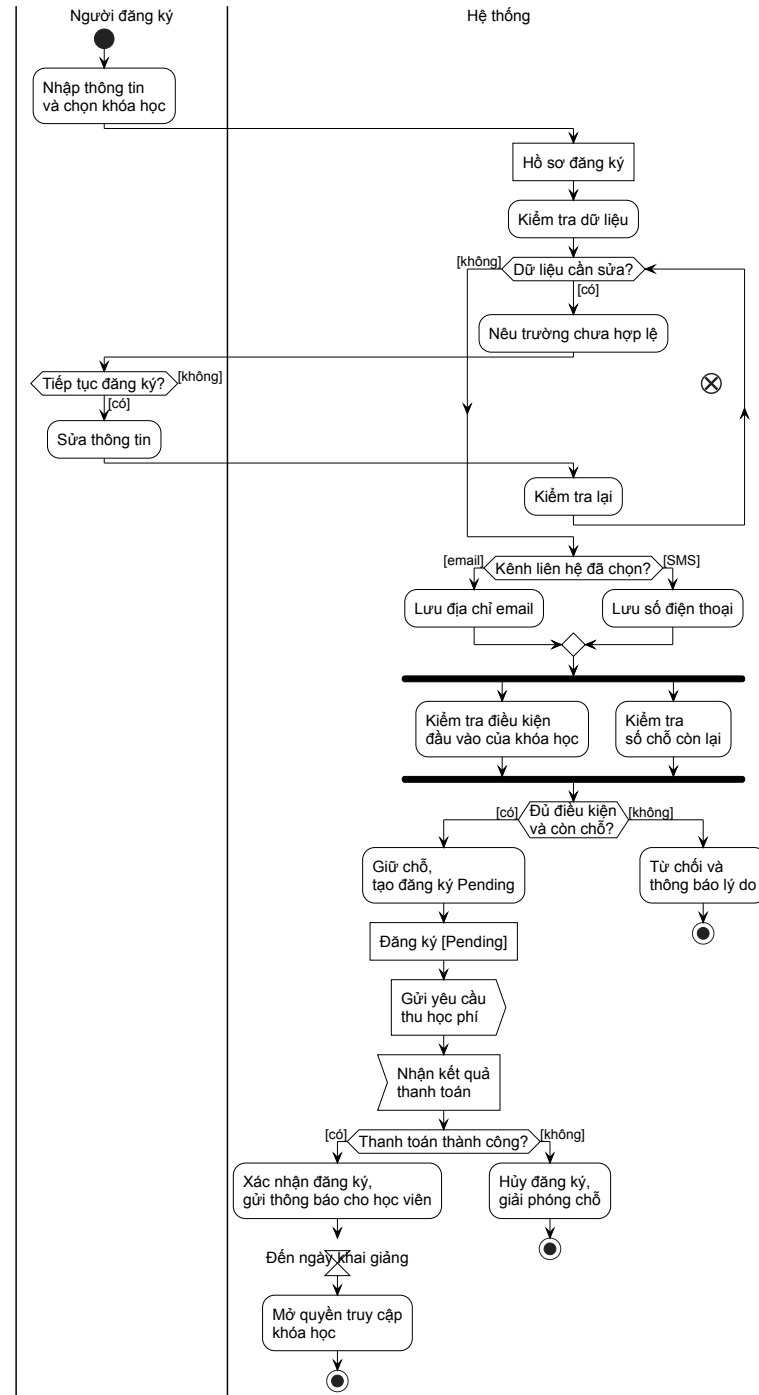
1 Bài toán và phạm vi

Một trung tâm đào tạo tiếp nhận đăng ký khóa học trực tuyến. Người đăng ký nhập hồ sơ và chọn khóa học. Hồ sơ chưa hợp lệ được trả về để sửa; người đăng ký có thể tiếp tục hoặc bỏ đăng ký.

Khi hồ sơ hợp lệ, hệ thống lưu kênh liên hệ đã chọn, kiểm tra điều kiện đầu vào và số chỗ còn lại. Nếu cả hai điều kiện đều đạt, hệ thống giữ chỗ, tạo đăng ký ở trạng thái **Pending** và gửi yêu cầu thu học phí. Thanh toán thành công dẫn đến xác nhận đăng ký. Đến ngày khai giảng, người học được mở quyền truy cập khóa học. Trường hợp không đủ điều kiện, hết chỗ hoặc thanh toán thất bại đều được kết thúc với kết quả tương ứng.

Ở ví dụ này, người đăng ký chọn đúng một kênh liên hệ: email hoặc SMS. Khóa học chưa khai giảng. Kết quả thanh toán được gửi về từ dịch vụ thanh toán sau khi yêu cầu thu học phí đã được tiếp nhận.

2 Sơ đồ hoàn chỉnh



Hình 46: Activity diagram của quy trình đăng ký khóa học

3 Đọc luồng chính

Bắt đầu từ chấm đen trong lane *Người đăng ký*. Sau bước nhập thông tin, hồ sơ được chuyển sang lane *Hệ thống* để kiểm tra. Nếu không cần sửa, luồng đi qua lựa chọn kênh liên hệ, sau đó đến thanh fork.

Fork mở hai nhánh kiểm tra: điều kiện đầu vào và số chỗ còn lại. Join đợi cả hai nhánh hoàn tất, rồi decision xét kết quả chung. Join không tự quyết định người đăng ký có đủ điều kiện hay không; nó chỉ đồng bộ các nhánh.

Nếu đủ điều kiện và còn chỗ, hệ thống tạo đăng ký **Pending**. Sau khi gửi yêu cầu thu học phí, luồng chờ sự kiện kết quả thanh toán. Nếu thanh toán thành công, hệ thống xác nhận đăng ký và gửi thông báo. Time event biểu diễn việc chờ đến ngày khai giảng. Sau bước mở quyền truy cập, activity final kết thúc quy trình.

4 Các nhánh và vòng lặp

Hồ sơ cần sửa

Decision của vòng lặp kiểm tra dữ liệu dẫn đến bước thông báo trường chưa hợp lệ. Người đăng ký có thể sửa và gửi lại. Sau lần kiểm tra lại, điều kiện vòng lặp được xét tiếp. Số lần sửa không được cố định trong ví dụ.

Người đăng ký không tiếp tục

Nhánh [không] ở câu hỏi “Tiếp tục đăng ký?” đi đến flow final. Nó dừng luồng đăng ký đang xét. Tại vị trí này chưa có các nhánh song song khác đang hoạt động, nên không còn công việc nào của lần đăng ký đó để tiếp tục.

Nếu đặt cùng flow final sau fork, nó chỉ dừng nhánh đi vào nút, không tự dừng các nhánh khác. Vì vậy không thể thay mọi activity final bằng flow final mà giữ nguyên ý nghĩa.

Chọn email hoặc SMS

Decision chọn một trong hai action lưu kênh liên hệ. Hai đường hội tụ tại merge trước fork. Merge cho luồng đã chọn đi tiếp; không đợi cả email lẫn SMS.

Đây là chỗ phân biệt rõ merge với join: người đăng ký chỉ chọn một kênh liên hệ, nhưng hệ thống cần kết quả của cả hai nhánh kiểm tra điều kiện và chỗ học.

Không đạt điều kiện hoặc hết chỗ

Sau join, guard [không] dẫn đến từ chối và thông báo lý do. Không tạo đăng ký Pending, không gửi yêu cầu thu học phí.

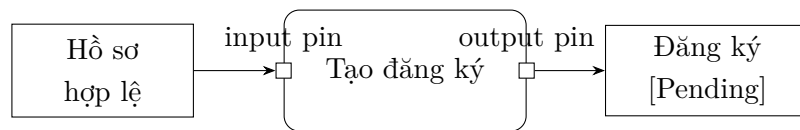
Thanh toán không thành công

Hệ thống hủy đăng ký và giải phóng chỗ đã giữ. Luồng kết thúc bằng activity final. Không được bỏ bước giải phóng chỗ vì trạng thái tài nguyên đã thay đổi trước khi thanh toán.

5 Dữ liệu, pin và các sự kiện

Hai object node trong Hình 46 là *Hồ sơ đăng ký* và *Đăng ký [Pending]*. Chúng biểu diễn dữ liệu đi qua quy trình, không phải thêm hai action “thực hiện hồ sơ” hoặc “thực hiện đăng ký”.

Có thể mở rộng riêng action tạo đăng ký để thấy rõ input pin và output pin như hình sau. Đây vẫn là dữ liệu của quy trình đăng ký khóa học, không phải một ví dụ khác.



Hình 47: Mở rộng action tạo đăng ký: object flow qua input pin và output pin

Input pin là nơi action nhận hồ sơ hợp lệ. Output pin là nơi action đưa ra đối tượng đăng ký mới. Các mũi tên giữa object node và pin là object flow. Những đường nối điều khiển thứ tự giữa các action trong sơ đồ tổng quan là control flow.

Send signal có đầu nhọn biểu diễn việc phát yêu cầu thu học phí. Accept event có phần lõm biểu diễn việc đợi kết quả. Hai việc này không có nghĩa dịch vụ thanh toán trả về ngay khi yêu cầu được gửi. Trong giải pháp thực tế, kết quả cần mang mã đăng ký để ghép với yêu cầu đang chờ.

Time event có hình đồng hồ cát biểu diễn việc đến ngày khai giảng. Nó khác “Nhận kết quả thanh toán”: một việc được kích hoạt theo thời gian, một việc được kích hoạt khi có sự kiện gửi đến.

6 Vì sao các ký hiệu được dùng như vậy?

Lane giúp xác định trách nhiệm, không phải hai chương trình chạy độc lập. Fork và join được dùng vì hai kiểm tra có thể thực hiện đồng thời nhưng phải hoàn tất trước khi xét điều kiện tạo đăng ký.

Việc kiểm tra còn chỗ trước đó chưa đủ để bảo đảm không vượt số chỗ khi nhiều người đăng ký cùng lúc. Action giữ chỗ phải kiểm tra và cập nhật nhất quán. Sơ đồ mô tả trách nhiệm cần thực hiện; thanh join không thay thế cơ chế kiểm soát cạnh tranh.

Ví dụ này dùng đủ các nhóm ký hiệu đã học: action, initial, control flow, object node và object flow, pin, decision và guard, merge, vòng lặp, fork-join, lane, hai loại nút kết thúc, send signal, accept event và time event.

V Ví dụ sequence diagram: đặt hàng và thanh toán

1 Bài toán và các thành phần

Một cửa hàng trực tuyến tiếp nhận đơn hàng, giữ tồn kho, xử lý thanh toán và chuyển tác vụ cho bộ phận giao hàng. Khách có thể dùng mã giảm giá và thử thanh toán tối đa ba lần. Nếu vẫn chưa thanh toán, cửa hàng giải phóng tồn kho đã giữ.

Khách hàng

Actor khởi tạo đặt hàng, thanh toán và yêu cầu hoàn tất checkout. Actor không trực tiếp cập nhật tồn kho hoặc trạng thái đơn.

Dịch vụ đơn hàng

Nhận các yêu cầu, kiểm tra dữ liệu, điều phối giữ hàng và thanh toán, ghi nhận kết quả, rồi gửi tác vụ giao hàng.

Kho hàng

Quản lý tồn kho và phần hàng đã giữ cho từng đơn. Trong hình, ký hiệu database dùng để biểu diễn thành phần lưu trữ; thông điệp `reserveItems` là tên trách nhiệm ở mức mô hình, không phải câu lệnh SQL.

Cổng thanh toán

Hệ thống bên ngoài xử lý yêu cầu thanh toán và trả kết quả cho phiên thanh toán.

Phiên thanh toán

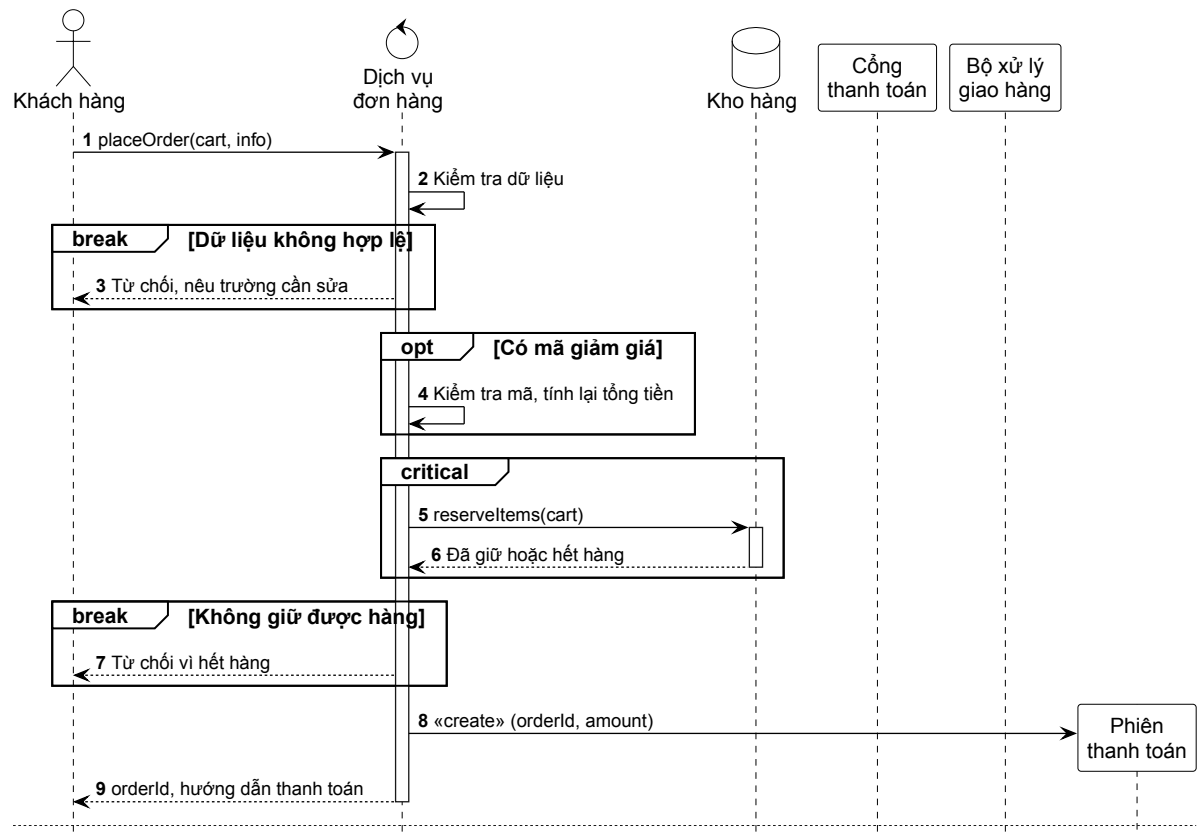
Đối tượng tạm thời được tạo khi đơn đủ điều kiện thanh toán. Nó được hủy sau khi khách hoàn tất checkout. Hủy phiên không có nghĩa hủy đơn hàng hay xóa giao dịch thanh toán đã ghi nhận.

Bộ xử lý giao hàng

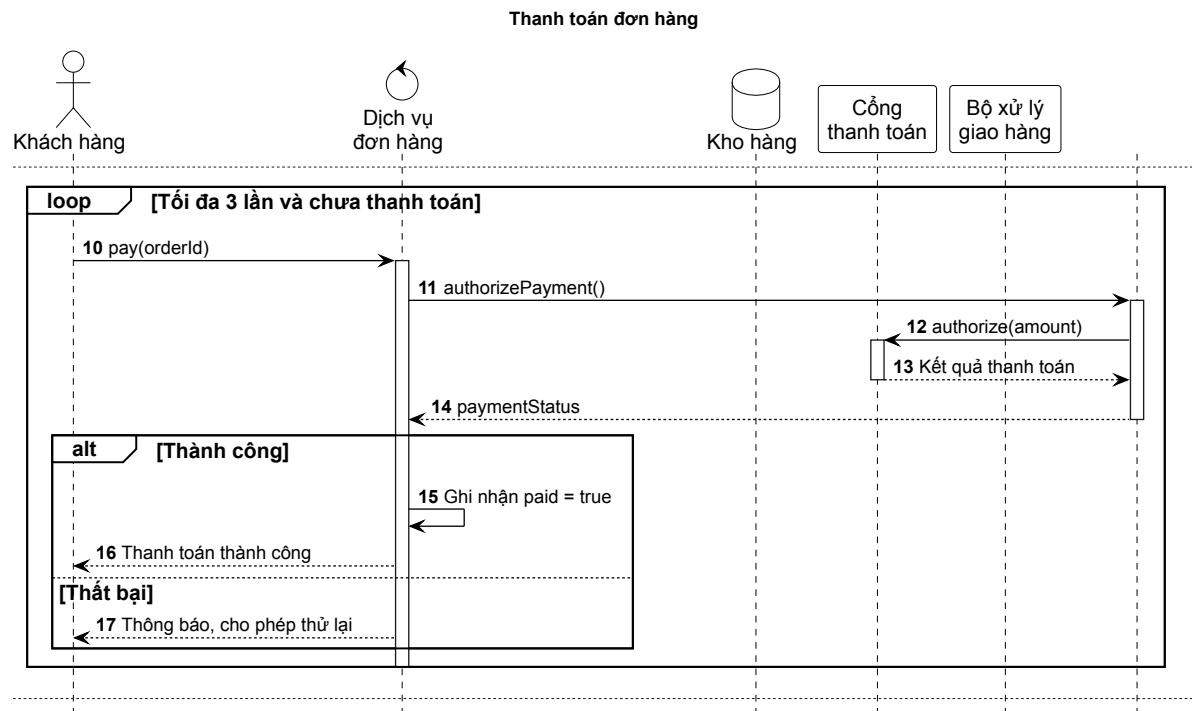
Nhận tác vụ bất đồng bộ và xử lý ở nền. Kết quả đặt hàng có thể được trả cho khách trước khi bộ xử lý này gửi thông báo tiếp nhận.

2 Sơ đồ hoàn chỉnh

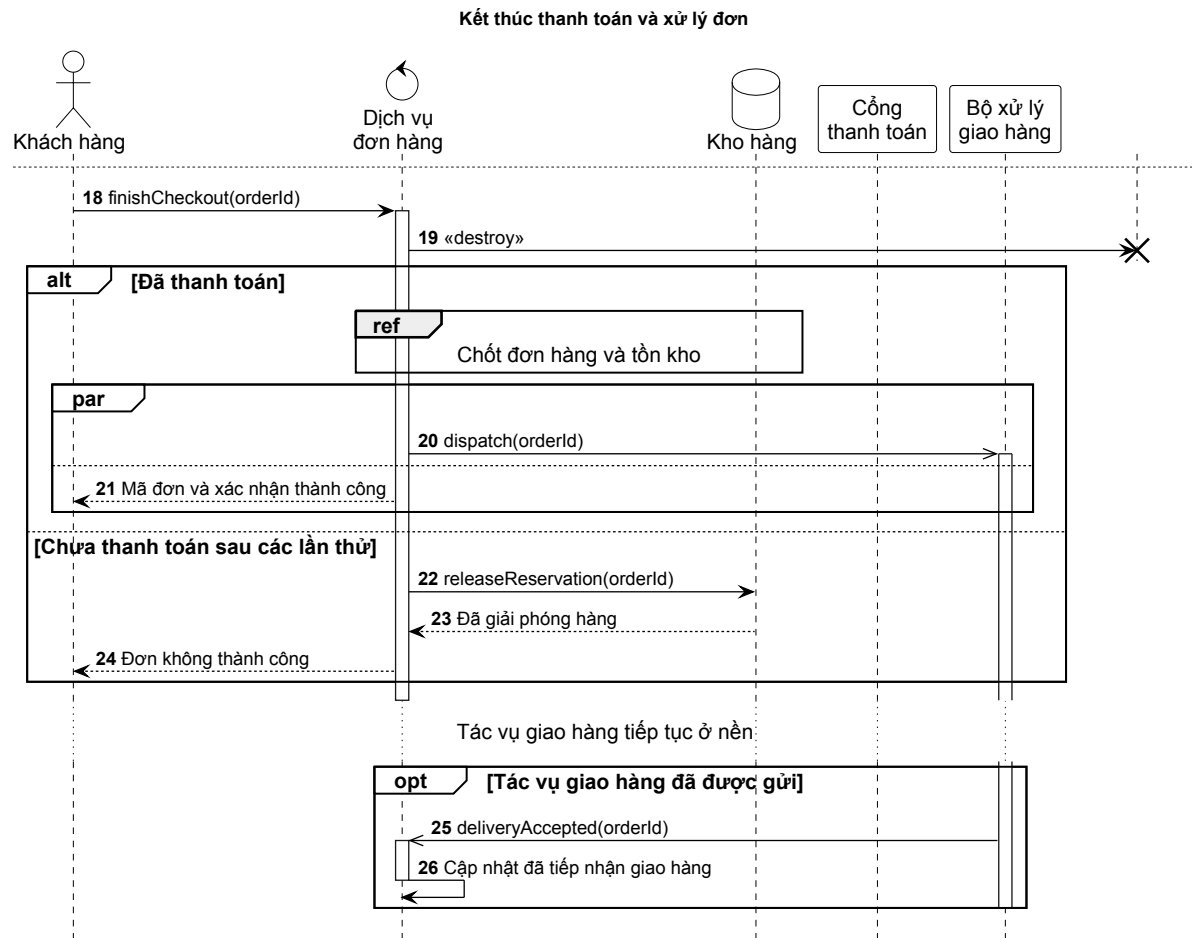
Cùng một interaction được chia thành ba hình nối tiếp để giữ cỡ chữ dễ đọc. Số thông điệp tiếp tục qua các hình; đường sống của các participant vẫn thuộc cùng một lần checkout.



Hình 48: Sequence đặt hàng: kiểm tra dữ liệu, giữ hàng và tạo phiên thanh toán



Hình 49: Sequence đặt hàng tiếp nối: vòng lặp thanh toán



Hình 50: Sequence đặt hàng tiếp nối: hoàn tất checkout và giao tác vụ ở nền

3 Đọc phần đặt hàng

Khách gọi `placeOrder`. Thanh activation của dịch vụ đơn hàng cho biết nó đang thực hiện yêu cầu này. Self-message kiểm tra dữ liệu thể hiện công việc nội bộ của dịch vụ, không phải lời gọi đến một thành phần mới.

Khung **break** đầu tiên xử lý dữ liệu không hợp lệ. Nếu điều kiện của khung đúng, dịch vụ trả thông báo lỗi và phần interaction bao quanh không tiếp tục sang giữ hàng hay thanh toán.

Khung **opt** chỉ chạy khi có mã giảm giá. Nếu không có mã, luồng đi qua phần này mà không thực hiện bước kiểm tra mã.

Khung **critical** bao quanh việc giữ hàng. Ý định của mô hình là tránh xen kẽ gây sai lệch đối với các lifeline được bao phủ trong đoạn này. Nó không tự tạo khóa hoặc transaction trong hệ thống thật; việc hiện thực phải chọn cơ chế bảo đảm giữ hàng nhất quán.

Nếu không giữ được hàng, **break** thứ hai kết thúc interaction với lý do hết hàng. Nếu giữ được, thông điệp `<<create>>` tạo phiên thanh toán. Đường sống của phiên bắt đầu ở vị trí nhận thông điệp tạo, không bắt đầu cùng hàng với các participant đã tồn tại.

Dịch vụ trả `orderId` và hướng dẫn thanh toán cho khách, sau đó kết thúc activation của lời gọi đặt hàng. Khách mới thực hiện các lời gọi thanh toán ở hình tiếp theo.

4 Đọc vòng lặp thanh toán

Khung **loop** cho phép tối đa ba lần thử, dừng sớm nếu đã thanh toán thành công. Điều kiện vòng lặp không yêu cầu lúc nào cũng thực hiện đủ ba lần.

Mỗi lần thử bắt đầu bằng lời gọi `pay`. Dịch vụ gọi phiên thanh toán; phiên gọi cổng thanh toán. Các mũi tên đồng bộ có đầu kín thể hiện bên gọi đợi lời gọi hoàn tất. Các đường nét đứt trả kết quả lần lượt từ cổng về phiên, từ phiên về dịch vụ và từ dịch vụ về khách.

Khung **alt** chọn nhánh thành công hoặc thất bại. Thành công thì ghi nhận đã thanh toán và trả xác nhận. Thất bại thì thông báo để khách có thể thử lại nếu còn lượt. Kết thúc mỗi lần thử, activation của dịch vụ cũng kết thúc.

Việc dịch vụ tự ghi nhận kết quả là self-message. Tên thông điệp diễn đạt trách nhiệm, không cần chép nguyên tên hàm của một ngôn ngữ cụ thể.

5 Đọc phần hoàn tất checkout

Khách gọi `finishCheckout`. Dịch vụ hủy phiên thanh toán tạm thời; dấu X chấm dứt đường sống của phiên. Những participant còn lại vẫn tồn tại.

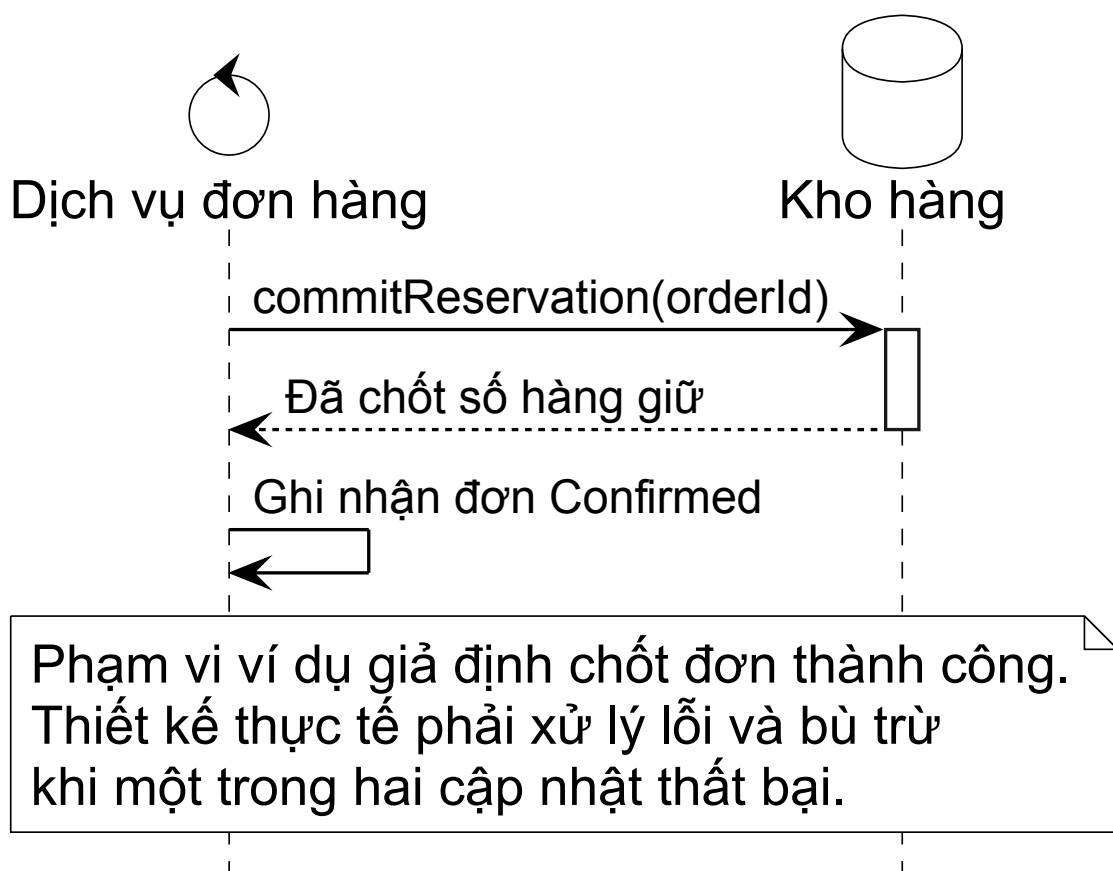
Khung **alt** xét kết quả đã thanh toán. Nhánh thành công tham chiếu interaction *Chốt đơn hàng và tồn kho* bằng **ref**. Nhánh chưa thanh toán giải phóng phần hàng đã giữ và trả kết quả không thành công.

Trong nhánh thành công, **par** có hai operand: gửi tác vụ giao hàng và trả xác nhận cho khách. Chúng không bị buộc vào một thứ tự toàn phần với nhau. Tuy nhiên, các sự kiện trên cùng một lifeline vẫn phải tuân theo thứ tự và các ràng buộc của interaction; **par** không có nghĩa mọi lệnh cùng xảy ra đúng một thời điểm.

Thông điệp **dispatch** dùng mũi tên bất đồng bộ đầu hở. Dịch vụ không đợi tác vụ giao hàng hoàn tất mới trả xác nhận. Sau khoảng thời gian xử lý ở nền, bộ xử lý gửi **deliveryAccepted** để dịch vụ cập nhật trạng thái tiếp nhận.

Khung **opt** cuối chỉ có ý nghĩa khi tác vụ giao hàng thực sự đã được gửi. Nhánh chưa thanh toán không phát sinh tác vụ đó nên cũng không được nhận một callback giao hàng không có nguồn gốc.

6 Mở rộng interaction được tham chiếu



Hình 51: *Interaction được gọi bằng ref: chốt đơn hàng và tồn kho*

Hình 51 triển khai phần **ref** ở Hình 50. Kho hàng chuyển phần hàng đã giữ thành phần hàng đã chốt cho đơn, trả kết quả, rồi dịch vụ cập nhật đơn thành đã xác nhận.

Ví dụ giả định bước chốt thành công. Nếu bước này có thể thất bại trong giải pháp thực tế, cần mô tả nhánh lỗi và cơ chế bù trừ phù hợp, không được mặc nhiên coi hai cập nhật ở hai nơi là một transaction duy nhất.

7 Liên hệ với từng ký hiệu đã học

Actor và participant cho biết ai tương tác. Lifeline cho biết vòng đời, activation cho biết đoạn thực hiện. Các lời gọi giữ hàng và thanh toán minh họa thông điệp đồng bộ; trả kết quả minh họa reply; **dispatch** và **delivery** minh họa bất đồng bộ.

Self-message thể hiện kiểm tra và ghi nhận nội bộ. Phiên thanh toán minh họa create và destroy. Các khung **alt**, **opt**, **loop**, **par**, **break**, **critical** và **ref** đều gắn với một tình huống cụ thể của lần checkout.

Không nên thêm một participant cho mọi bảng dữ liệu hoặc mọi màn hình. Một diagram tốt phải cho thấy sự phân chia trách nhiệm và trao đổi cần thiết, không phải liệt kê càng nhiều hình chữ nhật càng tốt.

VI Ví dụ state-chart diagram: vòng đời đơn sản xuất

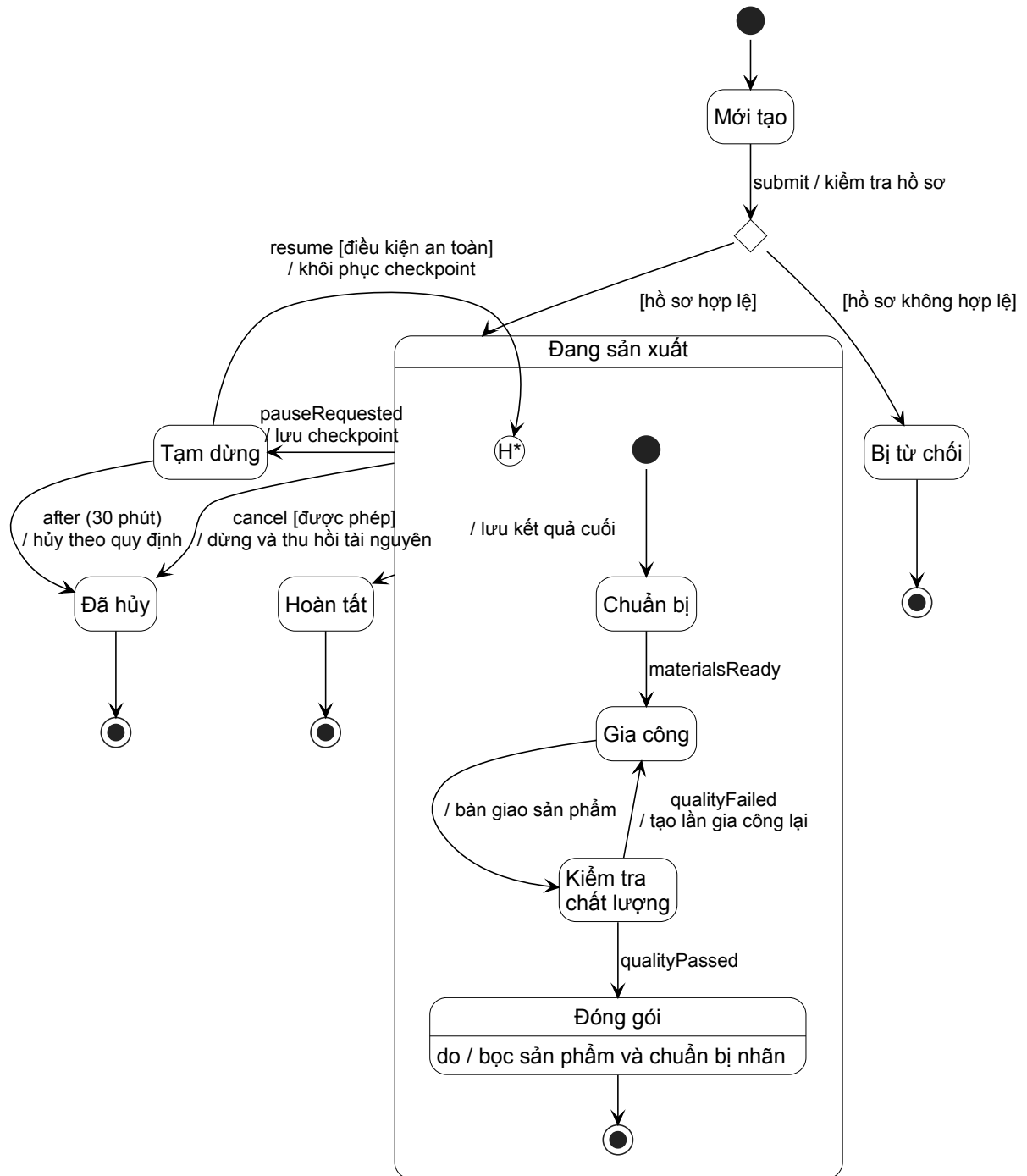
1 Đối tượng và các sự kiện

Đối tượng được mô hình hóa là **một đơn sản xuất trong xưởng**. State chart theo dõi đơn từ lúc tạo, kiểm tra hồ sơ, sản xuất đến khi hoàn tất, bị từ chối hoặc bị hủy.

Các bước lớn của sản xuất là chuẩn bị, gia công, kiểm tra chất lượng và đóng gói. Đơn có thể tạm dừng, tiếp tục đúng phần đang làm hoặc bị hủy. Trong gia công, máy có thể nhận tiến độ, hiệu chỉnh hoặc cần thay dụng cụ. Trong đóng gói, bọc sản phẩm và chuẩn bị nhãn có thể diễn ra song song.

Ba hình dưới đây là ba mức nhìn của *cùng một đơn*: hình tổng quan, mở rộng state Gia công và mở rộng state Đóng gói. Không phải ba máy trạng thái của ba đối tượng độc lập.

2 Sơ đồ tổng quan



Hình 52: State chart tổng quan của một đơn sản xuất

3 Đọc vòng đời và các nhánh kết thúc

Initial đưa đơn vào state *Mới tạo*. Sự kiện `submit` kích hoạt kiểm tra hồ sơ; effect trên transition ghi rõ việc kiểm tra được thực hiện trước khi choice xét kết quả.

Choice chọn nhánh theo guard. Hồ sơ không hợp lệ dẫn đến *Bị từ chối*; hồ sơ hợp lệ dẫn vào composite *Đang sản xuất*. Choice không phải một status cần lưu với tên “đang chọn”.

Khi vào composite theo đường thông thường, initial bên trong đưa đơn đến *Chuẩn bị*. `materialsReady` đưa đơn sang *Gia công*. Khi composite *Gia công* hoàn tất, completion transition bàn giao sản phẩm và đưa đơn sang *Kiểm tra chất lượng*; cấu trúc bên trong *Gia công* được mở rộng ở hình sau.

Chất lượng không đạt dẫn về *Gia công* để thực hiện một lần gia công lại. Đây là lần xử lý mới theo quy tắc của ví dụ, không phải mặc nhiên khôi phục vị trí máy ở lần gia công trước.

Chất lượng đạt dẫn đến *Đóng gói*. Khi hoạt động đóng gói hoàn tất, state con đi đến final của composite. Composite hoàn tất rồi kích hoạt completion transition sang *Hoàn tất*. Nhãn chỉ có phần effect “lưu kết quả cuối”, không có trigger sự kiện, vì đây là completion transition.

Sự kiện `cancel` chỉ đưa đơn sang *Đã hủy* khi guard [được phép] đúng. *Bị từ chối*, *Đã hủy* và *Hoàn tất* là ba kết quả nghiệp vụ khác nhau. Các final node sau chúng đều kết thúc máy trạng thái nhưng không làm ba kết quả trở thành một.

4 Tạm dừng và deep history

Sự kiện `pauseRequested` đưa đơn từ composite *Đang sản xuất* sang *Tạm dừng*, đồng thời lưu checkpoint. Transition đặt trên composite áp dụng khi composite đang hoạt động, không cần lặp cùng đường chuyển từ từng state con.

Nếu có `resume` và điều kiện an toàn đạt, đơn quay lại qua H^* . Deep history khôi phục cấu hình trạng thái sâu bên trong *Đang sản xuất*.

Ví dụ, trước khi tạm dừng, đơn ở *Đang sản xuất* / *Gia công* / *Đánh bóng*. Quay lại qua H^* sẽ về *Đánh bóng* thay vì bắt đầu lại từ *Chuẩn bị* hoặc *Cắt tạo hình*. Nếu tạm dừng trong *Đóng gói*, lịch sử sâu có thể nhớ trạng thái ở các region bên trong.

Điều kiện an toàn là guard, không phải sự kiện tự đến. Khi nhận `resume`, hệ thống mới xét guard trong transition này. Nếu điều kiện chưa đạt, transition đó không được thực hiện.

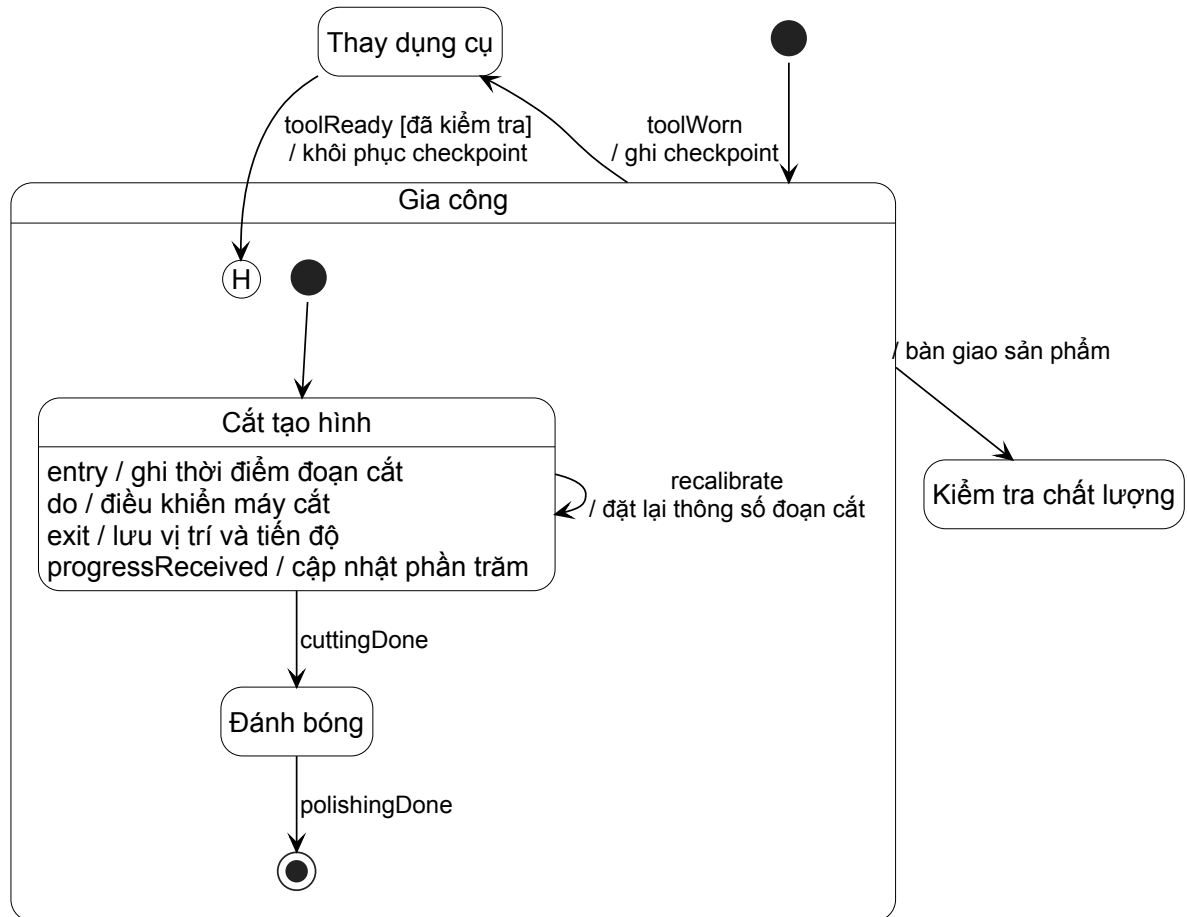
`after (30 phút)` là time event tính từ lúc vào state *Tạm dừng*. Nếu đơn vẫn còn ở state này sau khoảng thời gian đó, transition hủy được kích hoạt. Đây không phải lịch chạy cron lúc phút thứ 30 của mỗi giờ.

Checkpoint và history có trách nhiệm khác nhau: checkpoint lưu dữ liệu cần phục hồi, còn history nhớ vị trí điều khiển trong cấu trúc state. Thiếu dữ liệu checkpoint,



biết đang ở Đánh bóng chưa đủ để tiếp tục an toàn.

5 Mở rộng state Gia công



Hình 53: Mở rộng Gia công: hành vi trong state, self-transition và shallow history

Trong Hình 53, composite Gia công có hai state con: *Cắt tạo hình* và *Đánh bóng*. Khi bắt đầu gia công thông thường, initial dẫn vào *Cắt tạo hình*.

entry, do và exit

Vào *Cắt tạo hình* thì ghi thời điểm bắt đầu đoạn cắt. Trong thời gian state hoạt động, do điều khiển máy cắt. Rồi state thì **exit** lưu vị trí và tiến độ.

Internal transition

Sự kiện **progressReceived** cập nhật phần trăm bên trong *Cắt tạo hình*. Nó không làm đơn rời state nên không thực hiện lại **exit** và **entry**. Nếu cứ khởi động lại đoạn cắt mỗi khi nhận tiến độ thì thiết kế đã sai ý nghĩa nghiệp vụ.

External self-transition

Sự kiện **recalibrate** đi theo đường từ *Cắt tạo hình* trở lại chính nó. Vì đây là external self-transition, state bị rời rồi được vào lại: **exit** chạy, effect đặt lại thông số chạy, rồi **entry** chạy và hoạt động do được khởi động theo cấu hình mới.

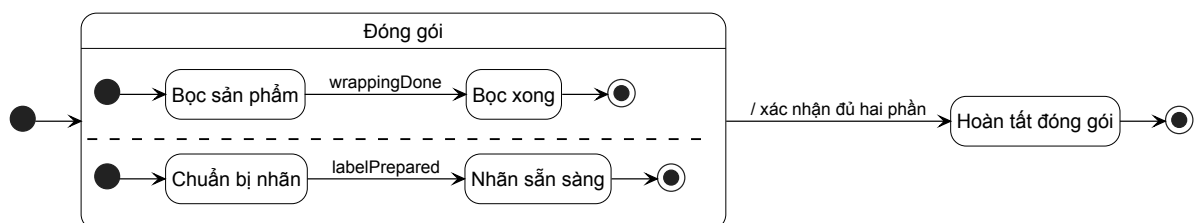
Thay dụng cụ và shallow history

Sự kiện **toolWorn** đưa đơn ra khỏi Gia công sang *Thay dụng cụ*, đồng thời ghi checkpoint. Sau khi dụng cụ sẵn sàng và đã được kiểm tra, transition **toolReady** đưa đơn vào H của Gia công.

Shallow history nhớ state con trực tiếp trước đó: *Cắt tạo hình* hoặc *Đánh bóng*. Vì hai state con này đơn giản, không có lớp state lồng sâu cần khôi phục, H là đủ cho tình huống này.

Khi **polishingDone** đưa *Đánh bóng* đến final bên trong Gia công, composite hoàn tất và chuyển sang Kiểm tra chất lượng. Hình mở rộng này không vẽ tiếp phần đóng gói vì phần đó đã xuất hiện trong sơ đồ tổng quan.

6 Mở rộng state Đóng gói



Hình 54: Mở rộng *Đóng gói* với hai *orthogonal regions*

Đóng gói có hai region được ngăn bằng đường nét đứt. Một region bọc sản phẩm; region còn lại chuẩn bị nhãn. Khi composite bắt đầu, cả hai region được khởi tạo.

Nếu đã bọc xong nhưng nhân chưa sẵn sàng, đối tượng vẫn còn ở composite Đóng gói. Final của region bọc không tự kết thúc region nhân. Chỉ khi cả hai region đạt final, composite mới hoàn tất.

Transition ra *Hoàn tất đóng gói* không cần thêm một sự kiện “cả hai đã xong” được gửi từ bên ngoài. Nó có thể là completion transition: khi các region đã hoàn tất, composite đủ điều kiện hoàn tất.

Hai vùng song song vẫn thuộc cùng vòng đời đơn sản xuất. Có hai region không có nghĩa ta đang ghép vòng đời độc lập của máy in nhân với vòng đời độc lập của khách hàng.

7 Tổng hợp ý nghĩa của ví dụ

Hình tổng quan thể hiện initial, state, transition, event, guard, effect, choice, composite, final, completion transition, time event và deep history.

Hình Gia công giải thích thêm entry-do-exit, internal transition, external self-transition và shallow history. Hình Đóng gói giải thích orthogonal regions cùng các initial và final của từng region.

Ba hình cần được đọc nhất quán: các hình mở rộng giải thích nội dung bên trong state đã có ở hình tổng quan. Không thêm một danh sách status mới không liên quan mỗi khi chuyển sang hình chi tiết.

Tài liệu tra cứu

Ý nghĩa ký hiệu có thể đối chiếu với [UML 2.5.1 của OMG](#). Cú pháp tạo hình được tra ở tài liệu chính thức của PlantUML: [Activity](#), [Sequence](#) và [State](#). Khi công cụ vẽ tự thêm hoặc thay cách hiển thị một ký hiệu, cần kiểm tra ý nghĩa UML thay vì chỉ nhìn hình có giống mẫu hay không.